

STUDENT PERFORMANCE PREDICTION

Machine Learning Based Web Application

Project Report

Predicting Student Exam Performance Using Machine Learning
Python • Pandas • Scikit-learn • Flask • HTML • CSS • JavaScript

Project Information	Details
Project Title	Student Performance Prediction
Project Type	Machine Learning / Predictive Analytics
Target Variable	Exam_Score
Primary Model	Linear Regression
Comparison Model	Random Forest Regressor
Web Framework	Flask
Dataset	Student Performance Factors
Prepared By	Soumyadip Porel
Academic Program	BCA

Abstract

Student performance is influenced by multiple academic, personal, environmental and educational factors. This project develops an end-to-end machine learning system that predicts a student's expected exam score from such factors. The project uses the Student Performance Factors dataset and follows a complete workflow: data inspection, missing-value handling, exploratory analysis, feature preparation, model training, evaluation, model selection, model serialization and web application integration.

Two regression algorithms were evaluated: Linear Regression and Random Forest Regressor. Using an 80/20 train-test split with `random_state=42`, Linear Regression achieved an R^2 score of 0.7083, MAE of 0.51 and RMSE of 2.05. Random Forest achieved an R^2 score of 0.6277, MAE of 1.13 and RMSE of 2.32. Based on R^2 , Linear Regression was selected as the best model for the deployed application.

The final system exposes the trained model through a Flask web interface where users enter student information and receive an estimated exam score together with a performance category and visual feedback. The project demonstrates how machine learning can be transformed into a practical, user-facing application.

1. Introduction

Educational institutions collect many kinds of information about students, including study habits, attendance, previous scores, motivation, parental involvement, access to resources, sleep, tutoring and other factors. Analysing these variables can help understand patterns in academic performance.

The purpose of this project is not to replace teachers or make high-stakes decisions. Instead, it demonstrates a predictive analytics approach that estimates exam performance from available student-related features and provides a simple interface for experimentation and learning.

2. Problem Statement

Build a machine learning application capable of predicting a student's exam score using academic, behavioural, family and school-related factors. The solution should include data preprocessing, model comparison, evaluation, saving of the trained model and deployment through a professional web interface.

3. Objectives

- Load and understand a real-world student performance dataset.
- Identify missing values, duplicates, data types and basic data-quality issues.
- Explore relationships between important student factors and exam performance.
- Separate numerical and categorical features for appropriate preprocessing.
- Handle missing input feature values using imputation.
- Encode categorical variables using one-hot encoding.
- Train and compare Linear Regression and Random Forest Regressor models.
- Evaluate models using MAE, RMSE and R^2 Score.
- Select and save the best-performing model using Joblib.
- Integrate the trained model with Flask.
- Develop a responsive and animated web interface for prediction.
- Create a project structure suitable for GitHub and portfolio presentation.

4. Project Scope

- Predictive modelling of Exam_Score as a continuous numerical value.
- Use of structured tabular student-performance data.
- Offline model training followed by local Flask deployment.
- Interactive prediction through a browser-based form.
- Educational and portfolio demonstration of an ML deployment workflow.

5. Dataset Description

The dataset contains 6,607 records and 20 columns. The target variable is Exam_Score. There are 18 predictor variables plus the target. The raw dataset contains missing values in several columns and no duplicate rows were identified during the project workflow.

Feature	Type	Description
Hours_Studied	Numerical	Study hours
Attendance	Numerical	Attendance percentage
Parental_Involvement	Categorical	Level of parental involvement
Access_to_Resources	Categorical	Access to educational resources
Extracurricular_Activities	Categorical	Participation in extracurricular activities
Sleep_Hours	Numerical	Average sleep hours
Previous_Scores	Numerical	Previous academic score
Motivation_Level	Categorical	Student motivation level
Internet_Access	Categorical	Internet access availability
Tutoring_Sessions	Numerical	Number of tutoring sessions
Family_Income	Categorical	Family income category
Teacher_Quality	Categorical	Teacher quality category
School_Type	Categorical	School type
Peer_Influence	Categorical	Peer influence category
Physical_Activity	Numerical	Physical activity level
Learning_Disabilities	Categorical	Learning disability indicator
Parental_Education_Level	Categorical	Parent education category
Distance_from_Home	Categorical	Distance category
Gender	Categorical	Gender category
Exam_Score	Target	Exam score to be predicted

6. Data Quality and Preprocessing

Initial dataset checks identified 259 missing cell values across the dataset and 0 duplicate rows. The target variable contained 2 missing Exam_Score records. Because a supervised regression model requires a valid target, rows with missing Exam_Score were removed before model training.

Check	Result / Treatment
Original records	6,607
Original columns	20
Missing cells	259
Duplicate rows	0
Missing target rows removed	2
Records used for modelling	6,605
Train/Test split	80% / 20%
Random state	42

For predictor variables, numerical missing values were handled with median imputation, while categorical missing values were handled using the most frequent category. Categorical variables were transformed with OneHotEncoder(handle_unknown='ignore'). The preprocessing operations were placed inside a

Scikit-learn ColumnTransformer and Pipeline so that the same transformations are automatically applied during prediction.

7. Exploratory Data Analysis

The observed Exam_Score values in the supplied dataset range from -65 to 101, with an average of 67.22. The exploratory analysis included target distribution, study-hours relationship, attendance relationship and a numerical correlation heatmap.

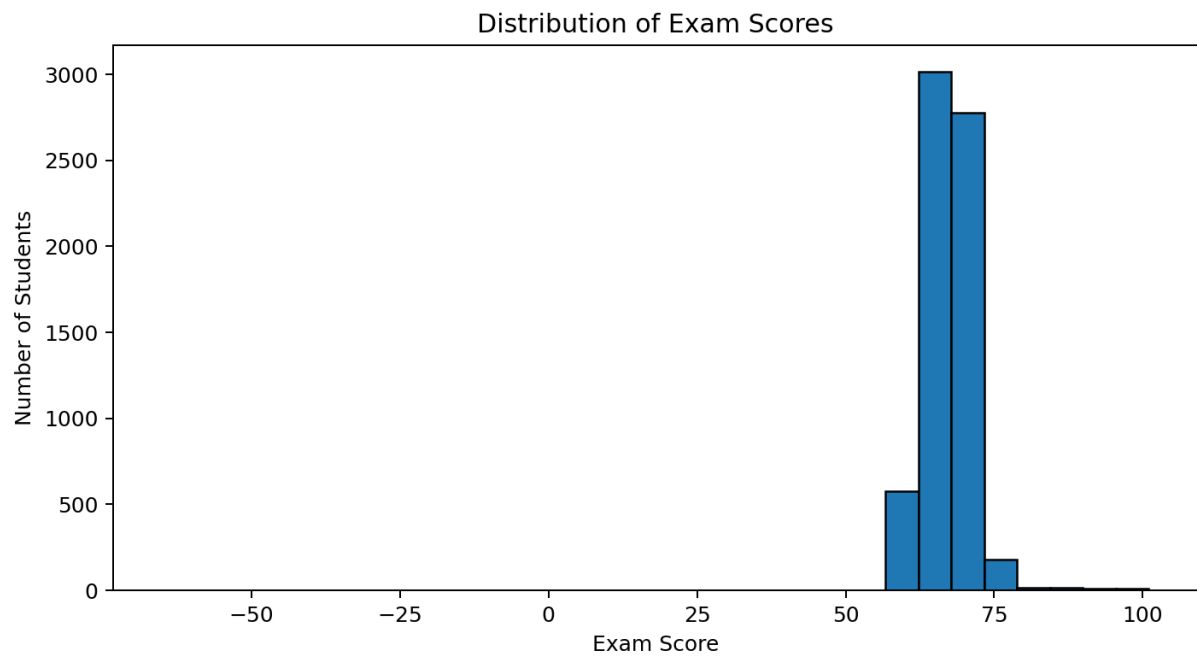


Figure 1. Distribution of Exam Scores

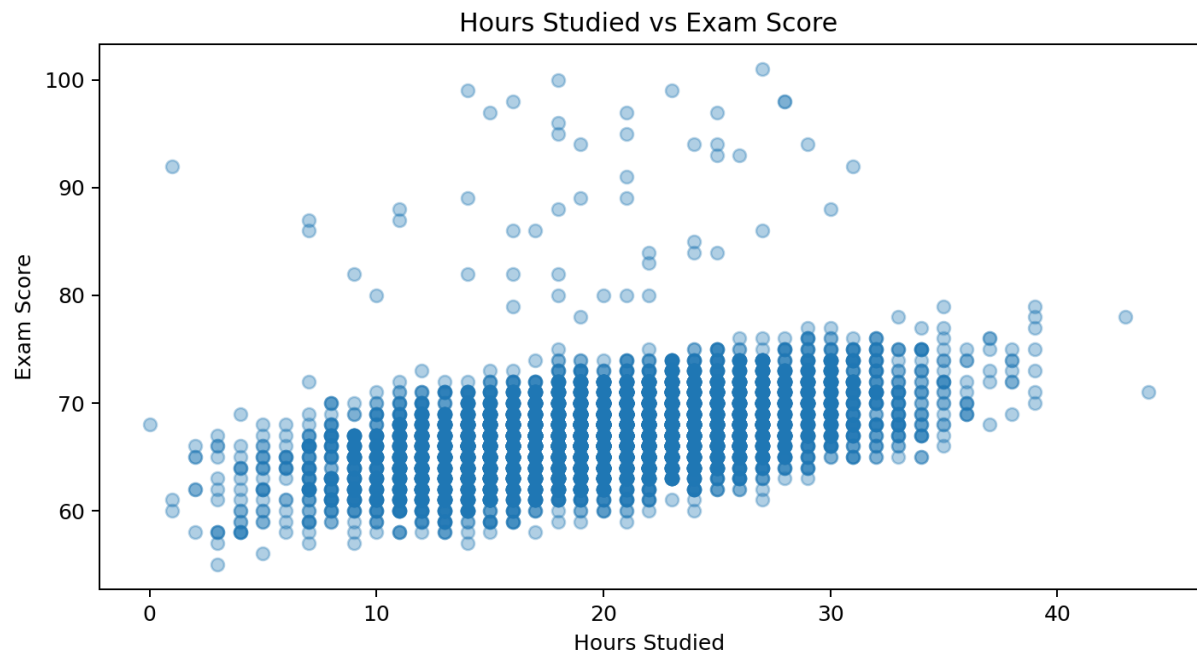


Figure 2. Hours Studied vs Exam Score

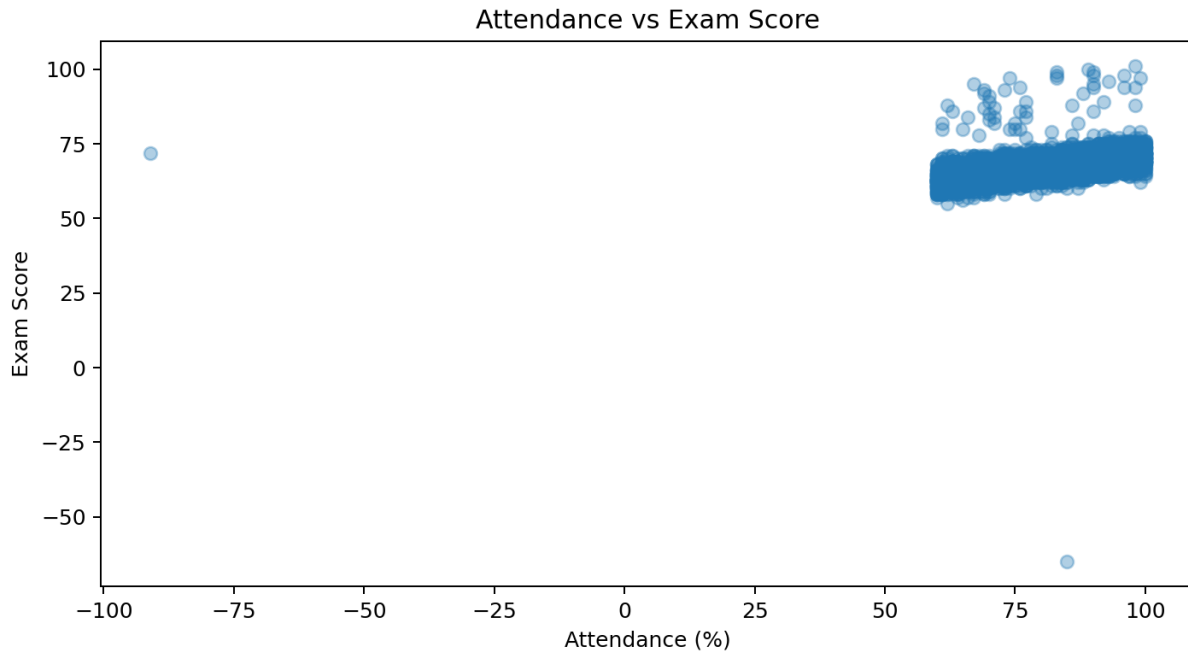


Figure 3. Attendance vs Exam Score

8. Methodology

1. Load the CSV dataset using Pandas.
2. Inspect dataset shape, columns, data types, missing values and duplicates.
3. Remove records where the target Exam_Score is missing.
4. Separate predictor variables (X) and target variable (y).
5. Identify numerical and categorical feature groups.
6. Create preprocessing pipelines for imputation and categorical encoding.
7. Split the data into 80% training and 20% testing subsets.
8. Train Linear Regression.
9. Train Random Forest Regressor with 200 estimators.
10. Generate predictions on the test set.
11. Calculate MAE, RMSE and R^2 Score.
12. Compare the models and select the model with the stronger R^2 performance.
13. Save the selected model as student_performance_model.pkl.
14. Load the saved model in Flask and expose a /predict endpoint.
15. Connect the HTML form and JavaScript frontend to the Flask prediction endpoint.

9. Machine Learning Models

9.1 Linear Regression

Linear Regression was used as a baseline regression algorithm. It estimates the relationship between the transformed input features and the continuous Exam_Score target. Its advantages include simplicity, fast training and interpretability.

9.2 Random Forest Regressor

Random Forest Regressor was used as a non-linear comparison model. It combines many decision trees and can capture more complex feature interactions. In this experiment, however, its test-set R^2 was lower than the Linear Regression model, so it was not selected for deployment.

10. Model Evaluation and Results

The models were evaluated using three standard regression metrics. MAE measures average absolute prediction error, RMSE penalizes larger errors more strongly, and R^2 measures the proportion of target variation explained by the model.

Model	MAE	RMSE	R^2 Score	Decision
Linear Regression	0.513	2.054	0.7083	Selected
Random Forest	1.132	2.320	0.6277	Not selected

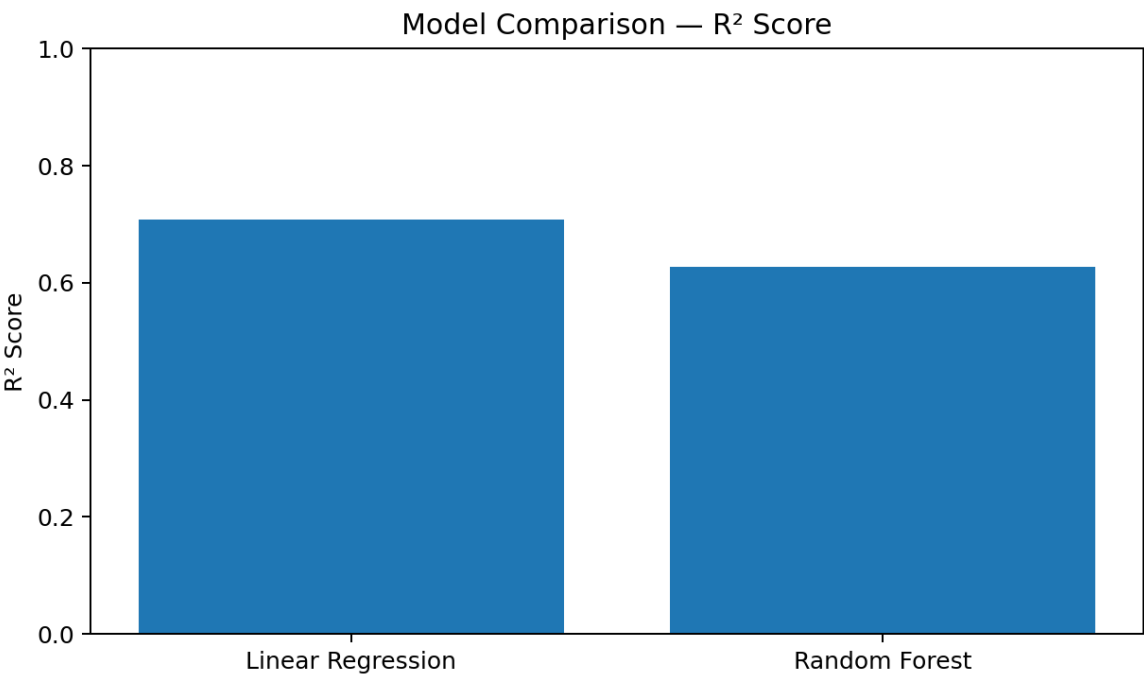


Figure 4. Model Comparison by R^2 Score

Linear Regression was selected because it produced the higher R^2 score (0.7083) and the lower RMSE (2.054) compared with Random Forest. Its MAE was also lower (0.513), indicating smaller average absolute prediction error on the test set.

11. Model Serialization

The selected preprocessing-and-model pipeline is saved using Joblib as `student_performance_model.pkl`. Saving the complete pipeline is important because it preserves both feature preprocessing and the trained regression model, allowing the Flask application to accept raw form values without manually reproducing the training transformations.

12. Web Application

The machine learning model is integrated into a Flask web application. The application contains a professional dark-themed interface with responsive layout, glass-style cards, animated elements, smooth navigation and an interactive prediction result.

Component	Technology / Purpose
Backend	Flask + Python
Model loading	Joblib
Input processing	Pandas DataFrame
Prediction API	POST /predict
Frontend	HTML5
Styling	CSS3
Interaction	JavaScript
Result visualization	Animated score/progress display

The user enters the same feature types used during model training. JavaScript sends the form values as JSON to the Flask /predict endpoint. Flask constructs a one-row Pandas DataFrame, passes it to the saved model pipeline and returns the predicted score as JSON. The frontend then displays the prediction and performance category.

13. Project Structure

```
Student_Performance_Prediction/
├── data/
│   └── StudentPerformanceFactors.csv
├── models/
│   └── student_performance_model.pkl
├── notebooks/
│   └── model_training.ipynb
├── reports/
├── templates/
│   └── index.html
├── static/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── script.js
│   └── images/
├── app.py
├── requirements.txt
├── .gitignore
└── README.md
```

14. Technologies and Libraries

Technology	Role
Python	Core programming language
Pandas	Data loading and tabular processing
NumPy	Numerical operations
Matplotlib	Data visualization
Seaborn	Correlation and statistical visualization
Scikit-learn	Preprocessing, models and evaluation
Joblib	Model serialization
Flask	Web backend and prediction API
HTML5	Web page structure

CSS3	Professional responsive styling and animation
JavaScript	Form submission and dynamic prediction results
Git / GitHub	Version control and project portfolio hosting

15. Testing

- Verified that the trained model can be loaded from the saved .pkl file.
- Verified that the Flask application is structured to serve the prediction page.
- Verified that the form field names correspond to the model's training feature names.
- Verified the /predict endpoint workflow from frontend JSON input to model prediction.
- Verified that prediction output is constrained to a practical 0–100 score range in the Flask application.
- Compared two regression algorithms using an unseen test subset.

16. Limitations

- The model is trained on a single dataset, so its performance may not generalize to every school, region or student population.
- The dataset contains some unusual target values outside the normal 0–100 exam-score interpretation; these should be investigated in a production-quality data-cleaning stage.
- The current application provides an estimate, not a guaranteed result.
- Model performance could change if the data distribution changes.
- The application currently focuses on prediction rather than longitudinal student tracking.

17. Future Improvements

- Add stricter data validation for impossible values such as Exam_Score outside the expected range.
- Add cross-validation and hyperparameter tuning.
- Compare additional models such as Gradient Boosting, XGBoost or HistGradientBoosting.
- Add feature-importance and explainability views.
- Store prediction history using SQLite.
- Add a teacher/admin dashboard with charts and summary statistics.
- Add authentication if the application is deployed for multiple users.
- Deploy the Flask application to a cloud platform.
- Add automated tests and CI using GitHub Actions.

18. Conclusion

The Student Performance Prediction project successfully demonstrates an end-to-end machine learning workflow, from raw tabular data to a working web application. The project performs data inspection and preprocessing, explores important relationships, trains multiple regression models, evaluates their performance and selects the stronger model based on objective metrics.

With an R^2 score of 0.7083, MAE of 0.513 and RMSE of 2.054 on the test set, Linear Regression was selected for the final application. The trained pipeline was saved using Joblib and connected to Flask, allowing users to enter student information and receive an estimated exam score through a browser-based interface.

Overall, the project provides practical experience in Python programming, data preprocessing, exploratory data analysis, supervised machine learning, model evaluation, model deployment, frontend development and GitHub-based project management.

19. References and Learning Resources

- Scikit-learn documentation — machine learning algorithms, preprocessing and evaluation.
- Pandas documentation — data loading and DataFrame operations.
- Flask documentation — Python web application development.
- Matplotlib and Seaborn documentation — data visualization.
- Joblib documentation — model persistence.
- Kaggle — Student Performance Factors dataset used for the project.